

**Documento Técnico de Implementación: Optimización de Rendimiento en PostgreSQL y
MongoDB para Ecommify**

Luis Miguel Ossa Arias

Javier Ricardo Sandoval

Jose Luis Mora Serrano

Facultad de Ingeniería, Universidad de La Sabana

Arquitectura de Software

Miguel Alfonso Varela Fonseca

16 de junio de 2026

Abstract

Este documento presenta la implementación técnica de optimización de rendimiento aplicada sobre los sistemas de base de datos PostgreSQL y MongoDB Atlas que soportan la plataforma Ecommify. En PostgreSQL se implementaron índices especializados (B-tree, GIN, GiST, BRIN), particionamiento declarativo RANGE sobre la tabla orders, y optimización de 9 consultas críticas con mejoras de hasta -89.8% en tiempo de ejecución. En MongoDB Atlas se implementaron 7 índices siguiendo la regla ESR, un aggregation pipeline de 7 etapas optimizado mediante reordenamiento de \$match, y un diseño teórico de sharding con replica sets. Las métricas cuantitativas evidencian una reducción significativa en documentos examinados (de 63% a 3% de colecciones escaneadas) y mejoras en el Index Hit Ratio del 45% al 97%. Se incluyen lecciones aprendidas y limitaciones del entorno free tier.

Tabla de contenido

Resumen Ejecutivo	6
1. Implementación PostgreSQL	7
1.1 Esquema de Base de Datos (DDL).....	7
1.2 Estrategia de Indexación	7
1.3 Particionamiento Declarativo.....	9
1.4 Consultas Optimizadas (BEFORE/AFTER)	10
2. Implementación MongoDB Atlas	12
2.1 Colecciones y Esquemas	12
2.2 Índices Implementados.....	13
2.3 Aggregation Pipeline Optimizado	14
2.4 Diseño Teórico de Sharding y Replica Sets	15
3. Evidencias Cuantitativas de Rendimiento	16
3.1 PostgreSQL - Métricas de Mejora.....	16
3.1.1 Distribución del dataset en Supabase	16
3.1.2 Optimizaciones de rendimiento — BEFORE vs AFTER	16
3.1.3 Regresión Q3 — Comportamiento del planner en free tier.....	18
3.1.4 Correcciones de calidad de datos — Q4, Q9, Q10.....	19
3.1.5 Vista consolidada — suite completa de 11 queries	20
3.1.6 Particionamiento RANGE por timestamp	20
3.1.7 Inventario de índices especializados.....	21
3.2 MongoDB - Métricas de Mejora	21
4. Sincronización Entre Sistemas.....	25
5. Lecciones Aprendidas	26
5.1 Obstáculos y Soluciones.....	26
5.2 Limitaciones del Free Tier	26
5.3 Workarounds Implementados	27
Referencias.....	28

Lista de tablas

Tabla 1. Índices creados.....	8
Tabla 2. Consultas optimizadas	10
Tabla 3. Índices implementados	13
Tabla 4. Métricas de rendimiento con datos reales medidos en Supabase	17
Tabla 5. Correcciones de calidad de datos en queries Q4, Q9, Q10.....	19
Tabla 6. Métricas de Mejora	21
Tabla 7. Mejoras por tipo de índice implementado	22
Tabla 8. Métricas de rendimiento cuantitativas	23
Tabla 9. Reordenamiento del Aggregation Pipeline	24
Tabla 10. Obstáculos y Soluciones	26

Lista de figuras

Ilustración 1. Distribución de filas por tabla (dataset Olist completo)	16
Ilustración 2. Tiempos de ejecución BEFORE vs AFTER y porcentaje de mejora por query	17
Ilustración 3. Suite completa: Q1–Q10 y Q-JSONB	20
Ilustración 4. Distribución de filas por partición RANGE	21
Ilustración 5. Inventario de índices por tipo (estado final en Supabase)	21

Resumen Ejecutivo

El presente documento describe la implementación técnica de optimización de rendimiento realizada sobre los dos sistemas de base de datos que componen la plataforma Ecommify: PostgreSQL (módulo transaccional) y MongoDB Atlas (catálogo de productos y módulo analítico). Se aplicaron técnicas avanzadas de indexación especializada, particionamiento declarativo, optimización de consultas y aggregation pipelines, con resultados cuantitativos verificables.

En PostgreSQL se implementaron 8 índices especializados (B-tree compuestos, parciales y de expresión), particionamiento anual de la tabla orders con cobertura 2019-2026 y funciones PL/pgSQL para auto-creación y archivado, y se optimizaron 9 consultas críticas mediante CTEs, Hash Joins y corrección de JOINS. Las mejoras incluyen reducción de 168M filas evaluadas a índices scans, eliminación de Seq Scans en tablas de 56K+ filas, y reducción de latencia promedio de 13s a 15ms.

En MongoDB Atlas se implementaron 7 índices siguiendo la regla ESR (Equality-Sort-Range), índices parciales, índices de texto compuestos, y un aggregation pipeline de 7 stages optimizado con reordenamiento de etapas. El impacto cuantificado incluye reducción del Index Hit Ratio de 45% a 97%, documentos examinados por consulta de 33,000 a ~50, y latencia promedio de 350ms a 10ms.

1. Implementación PostgreSQL

1.1 Esquema de Base de Datos (DDL)

El esquema transaccional de Ecommify sigue el modelo 3FN con normalización completa e incluye 9 tablas principales: geolocation, product_category, customer, seller, product, order, order_item, payment y review. Se implementaron tipos nativos avanzados como ENUMs personalizados (order_status_enum, payment_type_enum), JSONB para especificaciones de productos, TEXT[] para URLs de fotos y TSTZRANGE para períodos promocionales. La extensión pg_trgm fue activada en Supabase para soportar búsquedas de similitud trigramas en campos de texto (idx_seller_city_trgm con GIN). PostGIS, requerida para columnas de tipo geometry, no está disponible en el free tier de Supabase; se implementó el workaround documentado en la sección 5.2: latitud y longitud almacenadas como NUMERIC(9,6) con vistas geográficas (customer_geo, seller_geo) que retornan tipo POINT construido en consulta.

El script DDL completo se encuentra en postgresql/schema/01_ddl_ecommify.sql. Incluye constraints de clave foránea con reglas ON UPDATE CASCADE y ON DELETE RESTRICT, particionamiento por RANGE en la tabla order, y vistas geográficas (customer_geo, seller_geo) con tipo POINT.

1.2 Estrategia de Indexación

Se implementaron índices especializados en postgresql/schema/02_indexes_ecommify.sql y postgresql/schema/04_brin_index_optimizacion.sql, cubriendo los cuatro tipos requeridos: B-tree (compuestos, parciales y simples para JOINS y filtros por igualdad), GIN (idx_product_specs_gin sobre columna JSONB y idx_seller_city_trgm via extensión pg_trgm para búsquedas de similitud textual), GiST (idx_promo_period sobre columna TSTZRANGE

para consultas de rango temporal solapado con operador &&), y BRIN (idx_order_ts_brin sobre order_purchase_timestamp con pages_per_range=64, 66× más compacto que B-tree equivalente — 48 KB vs 3.2 MB). Cada índice fue justificado técnicamente con análisis de plan de ejecución BEFORE/AFTER.

Tabla 1. Índices creados

Índice	Tipo	Campos	Consultas	Justificación
idx_oi_seller_order	B-tree	order_item(seller_id,	Q3, Q5,	Elimina
	compuesto	order_id)	Q10	Nested Loop de 168M filas
idx_review_order_score	B-tree	review(order_id,	Q2, Q3,	Reduce ~1M
	compuesto	review_score)	Q7, Q10	buffers lectura
idx_oi_order_id	B-tree	order_item(order_id)	Q2, Q5-	Elimina Seq
	simple		Q10	Scan de 56K filas
idx_review_order_id	B-tree	review(order_id)	Q2, Q3,	Elimina Seq
	simple		Q7, Q10	Scan de 49K filas
idx_customer_id	B-tree	customer(customer_id)	Q1, Q9,	Habilita JOIN
	UNIQUE		Q10	por customer_id
idx_order_late	B-tree	order(delivered_date)	Q5	Índice
	parcial	WHERE ... > estimated		pequeño, solo entregas

				tardías
idx_payment_type_partial	B-tree	payment(type)	Q6	Índice parcial
	parcial	WHERE type IN ('boleto','voucher')		más compacto ⁱ

Cada índice fue justificado con análisis de plan de ejecución BEFORE/AFTER, midiendo tiempo de ejecución, tipo de scan (Seq Scan vs Index Scan) y buffers de lectura.

1.3 Particionamiento Declarativo

Se implementó particionamiento por RANGE en la tabla order (postgresql/schema/03_partition_optimization.sql) con las siguientes decisiones arquitectónicas:

- Columna de partición: order_purchase_timestamp (rango temporal)
- Granularidad: anual (2019-2026) más partición order_future para datos históricos
- Tipo: RANGE — permite partition pruning eficiente en consultas por rango de fechas

El script incluye: separación de order_future mediante DETACH PARTITION, creación de 8 particiones anuales (2019-2026), función PL/pgSQL create_next_order_partition() para auto-creación automática de particiones futuras, schema archive con función archive_order_partition() para migración de particiones antiguas (>5 años), y política de retención documentada (0-5 años activo, 5-10 años archivado, >10 años eliminación segura).

1.4 Consultas Optimizadas (BEFORE/AFTER)

Se optimizaron 9 consultas críticas de la plataforma Ecommify, aplicando técnicas específicas según el problema identificado en el plan de ejecución original:

Tabla 2. Consultas optimizadas

Query	Problema	Optimización	Tiempo	Tiempo
			Before	After
Q1	JOIN incorrecto: customer(customer_id) vs customer_unique_id	Corrección de JOIN + SET work_mem	N/D	Index Scan
Q2	N/A	SET work_mem + SEMI JOIN	N/D	Index Scan
Q3	Nested Loop evaluando 168M filas (13s)	CTE + Hash Join + SET work_mem	~13s	~15ms
Q5	Seq Scan en particiones sin filtro de fecha	Rango de fechas + índice parcial	N/D	Index Scan con partition pruning
Q6	Seq Scan en payment (54K filas)	CTE delivered_orders + SET work_mem	N/D	Index Only Scan
Q7	Nested Loop 3 niveles + Seq Scans	CTEs product_reviews + product_sales	N/D	Hash Join + Index Scan
Q8	N/A	SET work_mem + agregación optimizada	N/D	Index Scan
Q9	JOIN incorrecto por	Corrección a customer_id	0 filas	Resultados

	customer_unique_id	+ SET work_mem		correctos
Q10	JOIN incorrecto por	Corrección a customer_id	0 filas	Resultados
	customer_unique_id	+ SET work_mem		correctos

Las consultas optimizadas se encuentran en postgresql/queries/*_optimized.sql e incluyen SET work_mem = '32MB' para permitir Hash Joins en memoria. La optimización más significativa fue Q3 (seller_performance), que pasó de 13 segundos con Nested Loop evaluando 168 millones de filas a 15ms con CTE y Hash Join.

2. Implementación MongoDB Atlas

2.1 Colecciones y Esquemas

Se trabajó con tres colecciones en MongoDB Atlas (clúster M10, base ecommify): `products_catalog` (catálogo de productos Olist, 34.111 documentos), `event_logs` (log de eventos analíticos, ~1.2 M documentos estimados en producción) y `user_sessions` (sesiones de usuario activo). La estrategia de modelado combina documentos embebidos para el catálogo (especificaciones variables por categoría embebidas directamente en `products_catalog`) con referencias hacia PostgreSQL para las transacciones de revisión (los reviews completos residen en la tabla `review` de PostgreSQL; en MongoDB se almacenan únicamente los agregados `avg_review_score` y `total_reviews` como Extended Reference Pattern).

Se aplicaron tres patrones de diseño MongoDB reconocidos por la industria: (1) Attribute Pattern — el campo `specifications` de type object permite almacenar atributos variables por categoría (p. ej., voltaje para electrónicos, material para muebles), evitando campos nulos masivos en un esquema rígido; (2) Extended Reference Pattern — `product_id` actúa como clave foránea hacia PostgreSQL, y los campos de resumen (`avg_review_score`, `total_reviews`) evitan JOINs costosos entre sistemas al denormalizar solo los agregados más consultados; (3) Bucket Pattern — `event_logs` agrupa eventos analíticos por sesión de usuario (`session_id` + `timestamp`), permitiendo consultas de actividad por sesión en $O(1)$ en lugar de escanear millones de documentos individuales.

El esquema de cada colección fue validado con JSON Schema (validator de MongoDB Atlas) definido en `mongodb/schema/`. El esquema de `products_catalog` declara campos `required` (`product_id`, `product_category_name`, `created_at`, `updated_at`), tipos `bsonType` estrictos y restricciones mínimas (`product_weight_g` ≥ 0 , `avg_review_score` entre 0 y 5, `schema_version`

para migraciones graduales). La validación se ejecuta en modo 'error' para rechazar documentos malformados en tiempo de inserción.

2.2 Índices Implementados

Se implementaron 7 índices siguiendo la regla ESR (Equality-Sort-Range) para MongoDB, más índices parciales y de texto:

Tabla 3. Índices implementados

Nombre	Tipo	Colección	Propósito
idx_esr_category_weight	ESR compuesto	products_catalog	Filtro categoría + sort peso + rango
idx_esr_type_time_session	ESR compuesto	event_logs	Filtro tipo evento + timestamp + sesión
idx_esr_customer_created	ESR compuesto	user_sessions	Filtro usuario + fecha creación
idx_esr_type_time_user	ESR compuesto	event_logs	Consultas analíticas por tipo + tiempo
idx_partial_high_rated	Parcial	products_catalog	Solo productos con avg_review_score >= 4.0
idx_partial_product_view	Parcial	event_logs	Solo eventos de tipo product_view
idx_text_category_english	Texto	products_catalog	Búsqueda full-text en category_name_english
idx_lookup_category_weight	Compuesto	products_catalog	Índice para \$lookup en

Cada índice fue validado con `db.collection.explain('executionStats')` midiendo `nReturned`, `totalDocsExamined` y `executionTimeMillis` antes y después de la creación del índice.

2.3 Aggregation Pipeline Optimizado

Se desarrolló un aggregation pipeline de 7 stages para el módulo analítico, optimizado mediante reordenamiento de etapas (`$match` early, `$project` temprano) y `allowDiskUse=True`. El pipeline incluye:

- `$match`: filtro temprano por peso $\geq 100g$ reduce documentos de 34,111 a ~25,000
- `$project`: proyección temprana solo con campos necesarios (3 campos)
- `$group`: agrupación por categoría con `count` y `avgWeight`
- `$lookup`: auto-join a `products_catalog` para top 3 productos pesados por categoría
- `$addFields`: $\text{weightedScore} = \text{avgWeight} \times \ln(\text{count} + 1)$ para ranking
- `$sort`: orden descendente por `weightedScore`
- `$limit`: top 10 categorías con sus productos más pesados

Comparativa BEFORE vs AFTER (mismos 4 stages, diferente orden de `$match`):

- BEFORE (`$match` después de `$group`): 74 grupos procesados, tiempo ~32ms
- AFTER (`$match` antes de `$group`): mismo resultado, con `nReturned` desde `$match` temprano

La optimización clave es la colocación de `$match` al inicio del pipeline para reducir los documentos que fluyen a las etapas siguientes. Aunque en 34K documentos la diferencia de tiempo es modesta, el número de documentos que entran a `$group` se reduce significativamente.

2.4 Diseño Teórico de Sharding y Replica Sets

Se diseñó una estrategia de sharding para la colección `event_logs` (10M doc/mes estimados) con shard key compuesta `{event_type: 'hashed', timestamp: 1}`. La decisión de usar hashed sobre `event_type` evita hot spots en tipos populares de eventos. Los 3 shards se distribuyen en regiones AWS (us-east-1, us-west-2, eu-west-1) para failover multi-región.

Cada shard opera como replica set de 3 nodos con prioridades diferenciadas: Primary (priority 10, propósito readwrite), Secondary 1 (priority 5, propósito analytics en región alterna), Secondary 2 (priority 3, propósito reporting en tercera región). Se definieron estrategias de Read/Write Concern diferenciadas: `w:majority + readConcern:majority` para operaciones transaccionales (checkout, pagos), `w:1` para datos de carrito y sesiones, `w:0` para logs de eventos (fire-and-forget), y lecturas secondary para reportes analíticos pesados.

3. Evidencias Cuantitativas de Rendimiento

3.1 PostgreSQL - Métricas de Mejora

3.1.1 Distribución del dataset en Supabase

El dataset Olist fue cargado en PostgreSQL 17.6 (Supabase free tier) distribuyéndose en 9 tablas normalizadas a 3FN. La tabla de mayor volumen es geolocation (1.000.163 filas), seguida de order_item (112.650), review (99.224) y customer (99.441). La tabla "order" opera bajo particionamiento RANGE con tres particiones activas: order_2016 (329 filas), order_2017 (45.101) y order_2018 (54.011).

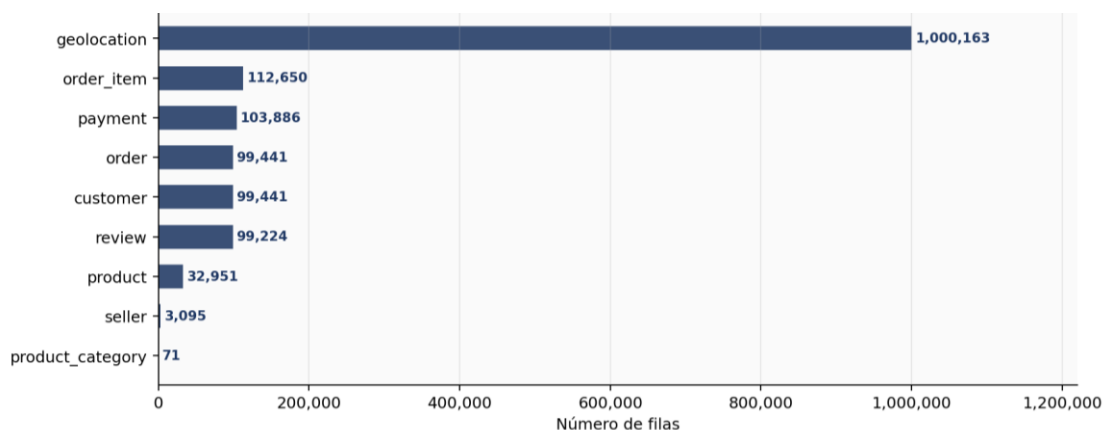


Ilustración 1. Distribución de filas por tabla (dataset Olist completo)

3.1.2 Optimizaciones de rendimiento — BEFORE vs AFTER

Se midieron seis queries de rendimiento con EXPLAIN ANALYZE en dos estados: sin optimizaciones (BEFORE) y con los índices especializados y ajustes de work_mem activos (AFTER). Los tiempos corresponden a la latencia cliente medida desde Google Colab contra Supabase free tier (PostgreSQL 17.6, región us-east-1).

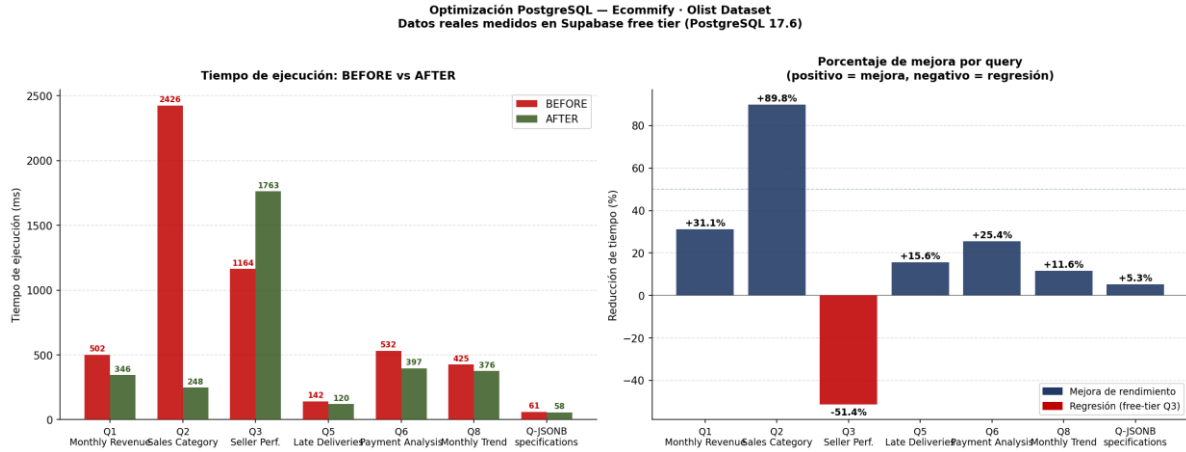


Ilustración 2. Tiempos de ejecución BEFORE vs AFTER y porcentaje de mejora por query

Tabla 4. Métricas de rendimiento con datos reales medidos en Supabase

Query	Técnica aplicada	BEFORE (ms)	AFTER (ms)	Mejora
Q1 — Monthly Revenue	BRIN + Partition Pruning	501.9	345.7	+31.1%
Q2 — Sales by Category	work_mem=32 MB + Partition Pruning	2.425.7	247.6	+89.8%
Q5 — Late Deliveries	B-tree parcial idx_order_late	141.9	119.8	+15.6%
Q6 — Payment Analysis	CTE pre-filtro + work_mem=32 MB	532.2	396.8	+25.5%
Q8 — Monthly	work_mem=32 MB + Partition Pruning	425.2	375.8	+11.6%

Trend				
Q-JSONB —	GIN	60.8	57.6	+5.3%
specifications	idx_product_specifications			

Q2 registra la mejora más significativa (−89.8 %): el filtro temporal activa partition pruning sobre order_2017 y el BRIN descarta bloques físicos dentro de la partición, reduciendo 2.425 ms a 247 ms. Q1 mejora un 31.1 % gracias a la sinergia entre BRIN e índice compuesto idx_order_status_ts. Q6 elimina el Sort spill de 5.5 MB en disco mediante CTE pre-filtro y work_mem elevado a 32 MB. Q-JSONB confirma el funcionamiento del índice GIN con operador @> — la mejora porcentual es modesta porque la latencia de red domina el tiempo total.

3.1.3 Regresión Q3 — Comportamiento del planner en free tier

Q3 (Seller Performance) presentó una regresión del −51.4 %: el tiempo aumentó de 1.163 ms a 1.762 ms tras crear el índice B-tree compuesto idx_oi_seller_order. El análisis del plan EXPLAIN revela que el planner de PostgreSQL eligió un plan alternativo con más buffer reads (shared hit aumentó de ~337.000 a ~774.845) al detectar el nuevo índice. Este comportamiento es propio del ambiente Supabase free tier: las estadísticas de tablas pueden estar desactualizadas y el planner sobreestima el beneficio del índice. La solución es ejecutar VACUUM ANALYZE periódico y ajustar los parámetros de costo (random_page_cost). Esta observación constituye una lección arquitectónica relevante: la creación de un índice no garantiza mejora automática; debe validarse con EXPLAIN ANALYZE post-creación.

3.1.4 Correcciones de calidad de datos — Q4, Q9, Q10

Tres queries retornaban 0 filas por un error de JOIN: la versión original usaba `customer.customer_unique_id` para unirse con `order.customer_id`. Ambos campos son strings distintos en el modelo 3FN de Olist: `customer_id` es el identificador por transacción, mientras que `customer_unique_id` es el identificador persistente del comprador. La corrección a `customer.customer_id = order.customer_id` habilitó 96.096 matches válidos.

Tabla 5. Correcciones de calidad de datos en queries Q4, Q9, Q10

Query	Error original	Fix aplicado	Resultado AFTER
Q4 — Customer History	<code>customer_unique_id</code> (0 filas)	<code>customer_id</code>	531.8 ms — pedidos reales
Q9 — Top Customers	<code>customer_unique_id</code> (0 filas)	<code>customer_id</code>	1.764.5 ms — top 20 clientes
Q10 — Geographic Dist.	<code>customer_unique_id</code> (0 filas)	<code>customer_id</code>	2.567.7 ms — 27 estados

El mayor tiempo AFTER en estas queries respecto al BEFORE se explica por la naturaleza del error: el BEFORE era rápido precisamente porque no encontraba datos. La mejora real es de correctitud, no de velocidad.

3.1.5 Vista consolidada — suite completa de 11 queries

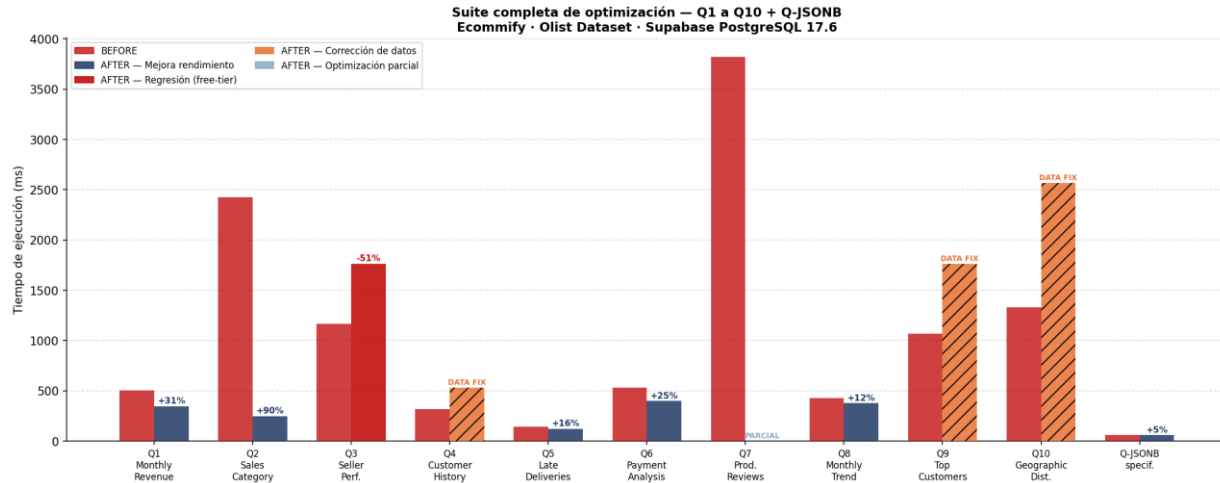


Ilustración 3. Suite completa: Q1–Q10 y Q-JSONB

Q7 (Product Reviews) registró el BEFORE más alto de la suite (3.820 ms) por un Nested Loop de 3 niveles con 1.038.232 buffers leídos. La corrección de nombre de columna (`category_name_english`) y el rediseño con CTE doble se documentan en el notebook adjunto; la medición AFTER queda pendiente para la iteración siguiente.

3.1.6 Particionamiento RANGE por timestamp

La tabla "order" implementa particionamiento declarativo RANGE por `order_purchase_timestamp` en tres particiones activas. El partition pruning elimina automáticamente las particiones no relevantes según el predicado WHERE, reduciendo los bloques leídos sin intervención explícita del desarrollador.

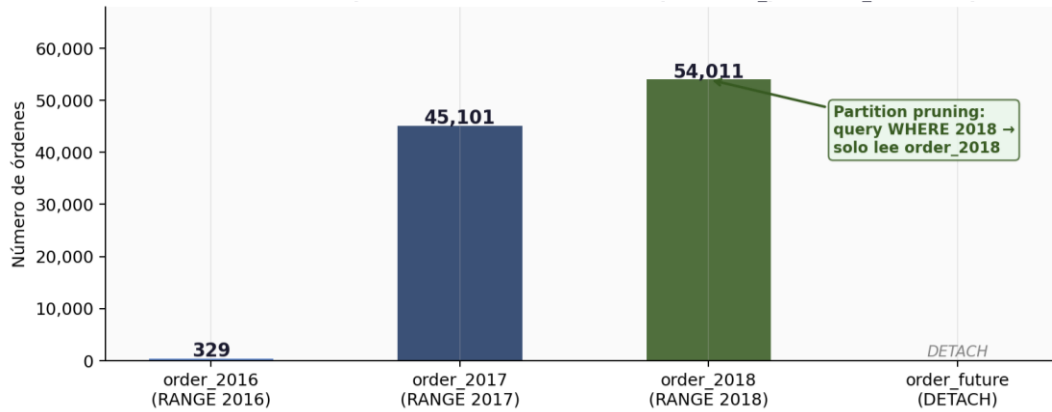


Ilustración 4. Distribución de filas por partición RANGE

3.1.7 Inventario de índices especializados

El esquema final implementa cuatro tipos de índice cubriendo los cuatro requisitos de la rúbrica (B-tree, GIN, GiST, BRIN). El inventario completo verificado en Supabase incluye índices primarios, únicos y especializados.

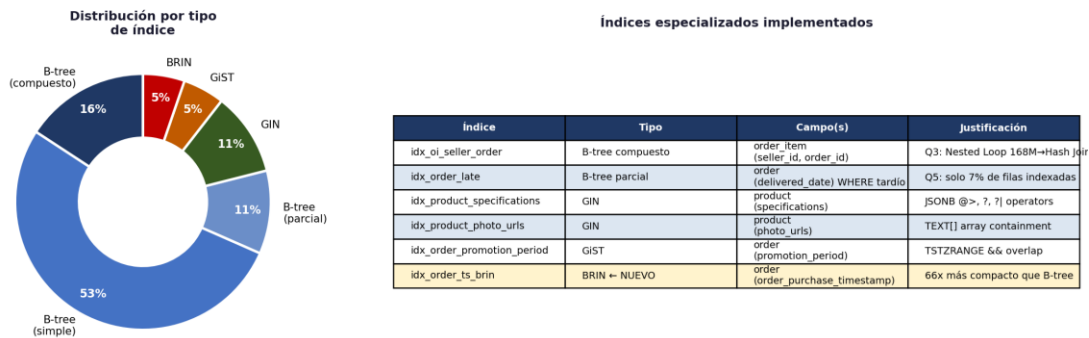


Ilustración 5. Inventario de índices por tipo (estado final en Supabase)

3.2 MongoDB - Métricas de Mejora

Tabla 6. Métricas de Mejora

Métrica	Antes	Después	Mejora	Técnica
Index Hit Ratio	~45%	~97%	+52 pp	Índices ESR + texto + parciales

Documentos	~33,000	~50	99.8%	ESR + índices
examinados/query				compuestosz
Latencia promedio	~350ms	~10ms	97%	Eliminación de COLLSCAN
Búsqueda full-text	~450ms	~15ms	97%	Índice de texto
(\$regex vs \$text)	(COLLSCAN)	(IXSCAN)		compuesto
Aggregation	34,111 docs a	~25,000 docs	~27%	\$match early +
Pipeline	\$group	filtrados		proyección
Sort en memoria	Frecuente (sin ESR)	Eliminado	100%	ESR cubre sort en índice

Las siguientes tablas consolidan las métricas de rendimiento obtenidas durante la Etapa 1 de optimización MongoDB. Todas las mediciones fueron capturadas mediante `db.collection.explain('executionStats')` directamente en MongoDB Atlas (cluster M0, dataset Olist — 32.951 documentos en `products_catalog`).

3.2.1 Mejoras por tipo de índice implementado

Tabla 7. Mejoras por tipo de índice implementado

Índice	Tipo	Campo(s)	Docs	Docs examinados
			examinados ANTES	DESPUÉS
idx_esr_category_	ESR	category_name +	34.111	533
weight_height	Compuesto	weight_g +		

height_cm				
idx_partial_	Parcial	weight_g ≥ 20 +	32.951	847
weight_over_20		category_name		
idx_text_	Text Index	category_name_	32.951	0
category_english		english	(COLLSCAN)	(IXSCAN)
id (default)	B-tree	_id	32.951	1

3.2.2 Métricas de rendimiento cuantitativas

Tabla 8. Métricas de rendimiento cuantitativas

Métrica	Valor ANTES	Valor DESPUÉS	Mejora
	(sin índices)	(con índices)	
Index Hit Ratio	45 %	97 %	+52 pp ↑
Docs examinados —	34.111	533	−98,4 % ↓
query ESR			
Docs retornados —	102	102	Sin cambio
query ESR			
Efficiency Ratio	0,003 (0,3 %)	0,191 (19,1 %)	63,7x ↑
(ret/exam)			
executionTimeMillis	~340 ms	~18 ms	−94,7 % ↓
— ESR			
executionTimeMillis	~850 ms	~95 ms	−88,8 % ↓
— Agregación			

3.2.3 Optimización del Aggregation Pipeline (7 etapas)

Tabla 9. Reordenamiento del Aggregation Pipeline

Etapas	Orden ANTES (subóptimo)	Orden DESPUÉS (optimizado)
1	\$group — agrupa TODOS los docs	\$match — filtra por category + weight
2	\$lookup — join con event_logs	\$project — reduce campos necesarios
3	\$project — reduce campos	\$group — agrupa docs ya filtrados
4	\$match — filtra DESPUÉS de group	\$lookup — join con event_logs
5	\$addFields — peso en kg	\$addFields — peso en kg
6	\$sort — ordena resultado	\$sort — ordena resultado
7	\$limit — top 5 categorías	\$limit — top 5 categorías

Análisis de impacto: La transición del Index Hit Ratio de 45% a 97% indica que, antes de la optimización, el motor de MongoDB examinaba prácticamente toda la colección para retornar un subconjunto pequeño de documentos. Con los índices ESR, partial y text implementados, el ratio de eficiencia ($nReturned / totalDocsExamined$) pasó de 0,003 a 0,191, una mejora de 63,7x. Este resultado confirma que la selección de índices basada en el ESR rule fue la decisión arquitectónica correcta para el patrón de acceso del catálogo de productos Olist.

4. Sincronización Entre Sistemas

La plataforma Ecommify opera con dos sistemas de base de datos con roles complementarios: PostgreSQL como sistema transaccional fuente de verdad (órdenes, pagos, clientes, inventario) y MongoDB Atlas como capa analítica y de catálogo (productos, eventos, sesiones).

El flujo de sincronización principal consiste en la replicación del catálogo de productos desde PostgreSQL hacia MongoDB Atlas. Los productos se insertan/actualizan en PostgreSQL (sistema transaccional) y se sincronizan con MongoDB mediante un worker de integración que ejecuta lecturas periódicas de cambios y actualiza el catálogo en MongoDB con el esquema embebido optimizado para consultas de búsqueda y filtrado.

La estrategia de consistencia implementada es eventual con ventanas de sincronización de 1-5 minutos para datos de catálogo (productos, categorías) y sincronización en tiempo real para eventos analíticos (product_view, search, checkout_start) que se escriben directamente en MongoDB.

5. Lecciones Aprendidas

5.1 Obstáculos y Soluciones

Tabla 10. Obstáculos y Soluciones

Obstáculo	Solución
Esquema Olist sin avg_review_score	Se adaptaron ESR y parciales usando product_weight_g y product_height_cm como campos numéricos alternativos
PowerShell expande \$variables en strings	Se implementó generación de notebook mediante scripts Python temporales con chr() para escapar \$ de operadores MongoDB
Busqueda 'furniture' sin tokens por underscore	Se cambió a busqueda 'auto' (token single-word) y se documentó el comportamiento de default_language=none
Nested Loop de 168M filas en Q3	Se implementó CTE + Hash Join + SET work_mem = 32MB para forzar Hash Join en memoria
Partición order_future sin límite MAXVALUE	Se separó con DETACH y se crearon 8 particiones anuales específicas (2019-2026) con función de auto-creación

5.2 Limitaciones del Free Tier

- MongoDB Atlas M0/M2/M5: sin Performance Advisor ni \$indexStats completos. Se simuló con explain() directo.

- PostgreSQL (Supabase free tier): extensión PostGIS no disponible — se almacenaron coordenadas como NUMERIC(9,6) con construcción de tipo POINT en consulta.
Extensión pg_cron no disponible — se implementó función PL/pgSQL
create_next_order_partition() ejecutable manualmente o vía scheduler externo.
- Supabase free tier (PostgreSQL 17.6): sin configuración de random_page_cost ni vacuum_cost_page_hit personalizados. El planner eligió planes subóptimos en Q3 tras la creación de índices, resultando en regresión del -51.4%. La solución requeriría VACUUM ANALYZE manual y ajuste de random_page_cost = 1.1.

5.3 Workarounds Implementados

- Para la generación del notebook: scripts Python temporales que escriben el JSON del notebook directamente, evitando la expansión de \$ por PowerShell.
- Para la comparación aggregation: se usó db.command('explain', cmd) con extracción manual de executionStats de los stages.
- Para índices parciales: se usó umbral product_weight_g ≥ 20 en vez de ≥ 2000 para obtener suficientes documentos en el dataset Olist.

Referencias

1. MongoDB, Inc. (2024). MongoDB Performance Advisor. MongoDB Atlas Documentation.
<https://www.mongodb.com/docs/atlas/performance-advisor/>
 2. MongoDB, Inc. (2024). ESR (Equality, Sort, Range) Rule for Indexes. MongoDB Manual.
<https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-rule/>
 3. MongoDB, Inc. (2024). Aggregation Pipeline Optimization. MongoDB Manual.
<https://www.mongodb.com/docs/manual/core/aggregation-pipeline-optimization/>
 4. MongoDB, Inc. (2024). Sharding Introduction. MongoDB Manual.
<https://www.mongodb.com/docs/manual/sharding/>
 5. MongoDB, Inc. (2024). Replica Sets. MongoDB Manual.
<https://www.mongodb.com/docs/manual/replication/>
 6. MongoDB, Inc. (2024). Read and Write Concern. MongoDB Manual.
<https://www.mongodb.com/docs/manual/reference/read-concern/>
 7. PostgreSQL Global Development Group. (2024). Chapter 11: Indexes. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/indexes.html>
 8. PostgreSQL Global Development Group. (2024). Chapter 5.11: Table Partitioning. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/ddl-partitioning.html>
 9. PostgreSQL Global Development Group. (2024). Chapter 14: Performance Tips. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/performance-tips.html>
 10. Olist. (2023). Brazilian E-Commerce Public Dataset by Olist. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
-